ウチダ電鉄　UchICa チャージ機能検証システム

概要

交通系 IC カード「UchICa」を券売機でチャージする機能を、コンソール画面で検証するシステムです。

- システム起動時のチャージ残高は 500 円とします。
- チャージは 1000 円単位で、一度に 10000 円までです。
- チャージを繰り返し、チャージ残高が 20000 円を超えたら、システムを終了します。
- 99 を入力するとチャージをキャンセルし、チャージ画面に戻ります。

制約事項

- 画面の表示内容、関数名、引数、戻り値は変更しないでください。数字や英字は半角で表示します。
- main 関数は「if __name__ == "__main__": main()」で実行します。
- ステップ 1（基本機能）が完成した後、ステップ 2（入力チェック）に進みます。
- ステップ 1 では、1〜10、99 だけが入力されるものとして作成します。（ステップ 1 では入力チェックをしなくていいということ）

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

画面仕様（ステップ 1）

起動するとすぐにチャージ画面を表示します。10、10 と入力した例です。（赤字は入力値、選択肢の表示は 2 回目以降を省略）

```
【チャージ機能】
チャージ残高は 500 円です。

1：1000 円
2：2000 円
3：3000 円
4：4000 円
5：5000 円
6：6000 円
7：7000 円
8：8000 円
9：9000 円
10：10000 円
チャージする金額を選択してください。（キャンセルする場合には 99 を入力）
10

10000 円チャージします。
チャージ残高は 10500 円です。


【チャージ機能】
チャージ残高は 10500 円です。
（選択肢は省略）
チャージする金額を選択してください。（キャンセルする場合には 99 を入力）
10

10000 円チャージします。
チャージ残高は 20500 円です。
チャージ残高が 20000 円を超えたため、システムを終了します。
```

◀ 現在のチャージ残高を表示する

◀ チャージ金額の選択肢を表示する

◀ 選択肢の番号を入力する

◀ チャージ金額とチャージ後の残高を表示し、チャージ画面に戻る

◀ 選択肢の番号を入力する

◀ 残高が 20000 円を超えたので、システムを終了する

起動直後に 99 を入力してキャンセルした場合は、残高を表示して、チャージ画面に戻ります。

```
チャージする金額を選択してください。（キャンセルする場合には 99 を入力）
99

チャージをキャンセルしました。
チャージ残高は 500 円です。


（以下、チャージ画面に戻る）
【チャージ機能】
チャージ残高は 500 円です。
```

◀ 99 を入力する

◀ キャンセルしても残高は表示する

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

関数仕様

関数名、引数と戻り値の数・順番・型、処理内容は変えないでください。

関数名	引数	戻り値	処理内容
main	なし	なし	チャージ残高を 500 円で初期化し、チャージ残高が 20000 円を超えるまで、charge 関数を繰り返し呼び出します。 20000 円を超えたら、「チャージ残高が 20000 円を超えたため、システムを終了します。」と表示して終了します。
charge	int balance （チャージ残高）	int 型 （チャージ後のチャージ残高）	チャージ画面を表示し、選択された金額をチャージ残高に加算して、チャージ後のチャージ残高を返します。キャンセルされた場合は、チャージ残高を変更せずに返します。 チャージした場合も、キャンセルした場合も、処理の最後にチャージ残高を表示します。 不適当な値が入力された場合は再入力を要求します。（この機能はステップ 2 で追加してください）

（型アノテーションを使う必要はありません）

関数実装の順番

次の順番で作成すると、1 つずつ動作を確認しながら進められます。

1. **charge 関数**（ステップ 1）
2. **main 関数**（ステップ 1）

なぜ charge 関数を先に作るのか？

main 関数は charge 関数を呼び出して使います。charge 関数が先に完成していれば（問題ないことが確認できていれば）、main 関数で問題が起きたときに「原因は main 関数側にある」と切り分けられます。慣れないうちは、他に依存しない部品（関数内で別の関数を呼び出していないもの）から作っていくことをお勧めします。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

charge 関数だけをテストする

main 関数が未完成でも、プログラムの最後を次のように書き換えれば、charge 関数だけを実行して確認できます。

```
if __name__ == "__main__":
    # main()
    print(charge(500))
```

確認する内容の例：

引数	入力	確認すること
500	3	「3000 円チャージします。」「チャージ残高は 3500 円です。」が表示され、3500 が返る
500	99	「チャージをキャンセルしました。」「チャージ残高は 500 円です。」が表示され、500 が返る
10500	10	「10000 円チャージします。」「チャージ残高は 20500 円です。」が表示され、20500 が返る。

※テストが終わったら、テスト用に追記した print(charge(500))は消し、必ず「main()」を実行する元の形に戻してください。

Copyright Uchida Human Development , Co., Ltd. All rights reserved.

画面仕様　入力チェック（ステップ 2）

以下の機能をステップ 1 のプログラムに追加してください。

チャージ金額の選択で、1～10、99 以外の値が入力された場合は、「正しい数値を入力してください。」と表示し、正しい値が入力されるまで再入力させてください。数字以外の文字列が入力された場合も同じです。今後、チャージ金額の選択肢が増えても、チェック処理を変えずに動作するよう工夫してください。（選択肢は常に 1000 円単位とします）

```
チャージする金額を選択してください。（キャンセルする場合には 99 を入力）
0
正しい数値を入力してください。
abc
正しい数値を入力してください。
3

3000 円チャージします。
チャージ残高は 3500 円です。
```

◀ 範囲外の値を入力

◀ 再入力を促す → 文字列を入力

◀ 再入力を促す → 正しい値を入力

◀ 通常のチャージ処理に進む

Copyright Uchida Human Development , Co., Ltd. All rights reserved.